

Structures de Données et Algorithmes Avancés

Lab S03 — Tables de hachage

Implémenter et observer une ChainedHashMap

Master 1 — 2025–2026

Dr. El Hadji Bassirou TOURÉ
École Supérieure Polytechnique
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Prérequis et contexte

Prérequis : CM S03 (fonction de hachage, chaînage séparé, facteur de charge, rehash). Lab S01 complété (Docker, Gradle, tests JUnit).

Objectif : Implémenter une `ChainedHashMap<K,V>` complète et *observer concrètement* ce qu'on a vu en CM : collisions, rehash, évolution du facteur de charge. Puis faire quelques exercices théoriques pour consolider.

Organisation : Ce lab est en **deux parties**. Partie I (pratique) : on code et on observe, pas à pas. Partie II (théorique) : des traces papier à dérouler en lien avec ce qu'on a codé.

Livrable : un dépôt Git avec le code + un rapport individuel (1–2 pages) décrivant ce que vous avez fait et appris.

À la fin de ce lab, vous saurez

1. Implémenter `ChainedHashMap<K,V>` avec chaînage séparé
2. Écrire des tests JUnit qui *forcent* des collisions
3. Ajouter un rehash automatique au seuil $\alpha > 0,75$
4. Visualiser la distribution des seaux et le déclenchement des rehashs
5. Dérouler manuellement une trace de chaînage et de sondage linéaire
6. Calculer $h(k) = |k.hashCode()| \bmod m$ et anticiper les collisions

Table des matières

I	Pratique : implémentation et observation	2
1	Environnement de travail	2
1.1	Créer le projet	2
2	Étape 1 — L'interface <code>Map<K,V></code>	2
3	Étape 2 — La classe <code>Paire<K,V></code>	3

4	Étape 3 — Squelette de ChainedHashMap	3
4.1	get et containsKey	4
4.2	put (version sans rehash)	4
4.3	remove	5
4.4	Compiler	5
5	Étape 4 — Tests JUnit : observer les collisions	5
6	Étape 5 — Ajouter le redimensionnement (rehash)	7
6.1	Test du rehash	8
7	Étape 6 — Observer : programme d’observation	9
8	Étape 7 — Observer : distribution des seaux	10
9	Git	11
II	Théorie : exercices d’application	12
10	Exercice 1 — Calcul de positions de hachage	12
11	Exercice 2 — Trace de chaînage séparé	12
12	Exercice 3 — Trace de sondage linéaire	14
13	Exercice 4 — Facteur de charge et rehash	14
14	Exercice 5 — Contrat equals ↔ hashCode	15
15	Pour aller plus loin (optionnel)	15
16	Aide-mémoire	16

Première partie

Pratique : implémentation et observation

1 Environnement de travail

Rappel Docker

Ce lab se fait dans l'environnement Docker du cours.

Lancement :

```
# Linux/macOS
docker run -it -v "$(pwd):/workspace" elbachirtoure/dsa-java-m1:v1.1

# Windows (Git Bash)
MSYS_NO_PATHCONV=1 docker run -it -v "$(pwd):/workspace" \
    elbachirtoure/dsa-java-m1:v1.1

cd /workspace
```

Si vous n'avez jamais utilisé cet environnement, revoyez le TP-1.

1.1 Créer le projet

```
cd /workspace
new-project lab-s03
cd lab-s03
```

La commande `new-project` crée un squelette Gradle avec JUnit 5 configuré, un `Main.java`, un `MainTest.java` et un dossier `src/main/java/`.

2 Étape 1 — L'interface `Map<K,V>`

Ce qu'on matérialise du CM

En S03 on a défini l'**ADT Map** comme un contrat (`put`, `get`, `remove`, `containsKey`, `size`). Ici on écrit ce contrat en Java, comme on l'a fait pour *Deque* en S01.

Créez le fichier `src/main/java/Map.java`.

```
1  /**
2   * ADT Map (dictionnaire associatif cle -> valeur).
3   * Chaque cle apparait au plus une fois.
4   */
5  public interface Map<K, V> {
6
7      /** Associe la valeur a la cle. Remplace si la cle existe deja.
8       * Retourne l'ancienne valeur, ou null si la cle etait absente. */
9      V put(K cle, V valeur);
10
11     /** Retourne la valeur associee, ou null si absente. */
12     V get(K cle);
13
14     /** Supprime la paire. Retourne la valeur, ou null si absente. */
15     V remove(K cle);
16 }
```

```

17  /** Teste l'appartenance de la cle. */
18  boolean containsKey(K cle);
19
20  /** Nombre de paires stockees. */
21  int size();
22
23  /** True si la map est vide. */
24  default boolean isEmpty() {
25      return size() == 0;
26  }
27 }

```

Pourquoi put retourne la valeur précédente ?

Cela permet de faire *en une seule opération* une mise à jour *et* de savoir si la clé existait. Sinon il faudrait appeler `containsKey`, puis `get`, puis `put` — trois fois plus coûteux.

3 Étape 2 — La classe Paire<K,V>

Créez `src/main/java/Paire.java` :

```

1  public class Paire<K, V> {
2      public final K cle;           // la cle ne doit JAMAIS changer
3      public V valeur;             // la valeur, elle, peut etre mise a jour
4
5      public Paire(K cle, V valeur) {
6          this.cle = cle;
7          this.valeur = valeur;
8      }
9
10     @Override
11     public String toString() {
12         return cle + "=" + valeur;
13     }
14 }

```

Pourquoi cle est final ?

Si on modifiait la clé *après* avoir inséré la paire, son hachage changerait et on ne pourrait plus la retrouver. La clé d'une paire, une fois insérée, ne doit jamais changer. C'est une règle générale des tables de hachage : **ne jamais mettre comme clé un objet dont l'état peut changer.**

4 Étape 3 — Squelette de ChainedHashMap

Ce qu'on matérialise du CM

Chaînage séparé : chaque case du tableau interne = une liste chaînée des paires qui collisionnent. La fonction de hachage $h(k) = |k.hashCode()| \bmod m$ envoie la clé dans une case.

Créez `src/main/java/ChainedHashMap.java`. On commence sans le rehash — on l'ajoutera à l'étape 5.

```

1  import java.util.LinkedList;
2

```

```

3 public class ChainedHashMap<K, V> implements Map<K, V> {
4
5     private LinkedList<Paire<K, V>>[] table;
6     private int m;           // taille du tableau (nombre de seaux)
7     private int n;           // nombre de paires stockees
8
9     private static final int CAPACITE_INITIALE = 8;
10
11     @SuppressWarnings("unchecked")
12     public ChainedHashMap() {
13         m = CAPACITE_INITIALE;
14         n = 0;
15         table = new LinkedList[m];
16         for (int i = 0; i < m; i++) {
17             table[i] = new LinkedList<>();
18         }
19     }
20
21     private int hash(K cle) {
22         return Math.abs(cle.hashCode()) % m;
23     }
24
25     public int size() { return n; }
26
27     public int capacity() { return m; } // utile pour observer
28
29     public double loadFactor() { // utile pour observer
30         return (double) n / m;
31     }
32 }

```

4.1 get et containsKey

```

1 public V get(K cle) {
2     int i = hash(cle);
3     for (Paire<K, V> p : table[i]) {
4         if (p.cle.equals(cle)) return p.valeur;
5     }
6     return null;
7 }
8
9 public boolean containsKey(K cle) {
10     int i = hash(cle);
11     for (Paire<K, V> p : table[i]) {
12         if (p.cle.equals(cle)) return true;
13     }
14     return false;
15 }

```

Pourquoi containsKey ne réutilise pas get ?

Si on faisait `return get(cle) != null;`, ça marcherait *sauf* si la clé est associée à la valeur `null` — la fonction retournerait alors `false` alors que la clé existe. On refait donc la boucle à la main.

4.2 put (version sans rehash)

```

1  public V put(K cle, V valeur) {
2      int i = hash(cle);
3      // Si la cle existe deja : mise a jour
4      for (Paire<K, V> p : table[i]) {
5          if (p.cle.equals(cle)) {
6              V ancienne = p.valeur;
7              p.valeur = valeur;
8              return ancienne;
9          }
10     }
11     // Sinon : nouvelle paire ajoutée en fin de liste
12     table[i].add(new Paire<>(cle, valeur));
13     n++;
14     return null;
15 }

```

4.3 remove

```

1  public V remove(K cle) {
2      int i = hash(cle);
3      var it = table[i].iterator();
4      while (it.hasNext()) {
5          Paire<K, V> p = it.next();
6          if (p.cle.equals(cle)) {
7              it.remove();
8              n--;
9              return p.valeur;
10         }
11     }
12     return null;
13 }

```

Pourquoi un itérateur explicite ?

On ne peut pas utiliser `for-each` pour supprimer un élément pendant le parcours (`ConcurrentModificationException`). L'itérateur explicite fournit `it.remove()` qui gère cela proprement.

4.4 Compiler

```
gradle --no-daemon build
```

À cette étape, la compilation doit passer sans erreur. Si vous avez un problème, vérifiez que vos trois fichiers (`Map.java`, `Paire.java`, `ChainedHashMap.java`) sont bien dans `src/main/java/`.

5 Étape 4 — Tests JUnit : observer les collisions

Ce qu'on matérialise du CM

Collisions inévitables : en CM on a démontré par le principe des tiroirs que deux clés différentes peuvent avoir le même hachage. Ici on va le provoquer et vérifier que notre chaînage fonctionne.

Créez `src/test/java/ChainedHashMapTest.java` :

```

1  import org.junit.jupiter.api.Test;
2  import static org.junit.jupiter.api.Assertions.*;
3
4  class ChainedHashMapTest {
5
6      @Test
7      void testPutGet() {
8          Map<String, String> carnet = new ChainedHashMap<>();
9          carnet.put("Awa Diop", "+221 77 555 01 23");
10         assertEquals("+221 77 555 01 23", carnet.get("Awa Diop"));
11     }
12
13     @Test
14     void testUpdate() {
15         Map<String, String> carnet = new ChainedHashMap<>();
16         carnet.put("Awa Diop", "+221 77 555 01 23");
17         String ancien = carnet.put("Awa Diop", "+221 77 999 00 00");
18         assertEquals("+221 77 555 01 23", ancien);
19         assertEquals("+221 77 999 00 00", carnet.get("Awa Diop"));
20         assertEquals(1, carnet.size()); // pas d'augmentation de taille
21     }
22
23     @Test
24     void testMissingKey() {
25         Map<String, String> carnet = new ChainedHashMap<>();
26         assertNull(carnet.get("Fatou Sall"));
27         assertFalse(carnet.containsKey("Fatou Sall"));
28     }
29
30     @Test
31     void testRemove() {
32         Map<String, String> carnet = new ChainedHashMap<>();
33         carnet.put("Awa Diop", "+221 77 555 01 23");
34         String supprimee = carnet.remove("Awa Diop");
35         assertEquals("+221 77 555 01 23", supprimee);
36         assertFalse(carnet.containsKey("Awa Diop"));
37         assertEquals(0, carnet.size());
38     }
39
40     /** Test qui FORCE une collision avec un hashCode custom. */
41     @Test
42     void testForcerUneCollision() {
43         Map<ClePerso, String> map = new ChainedHashMap<>();
44         // Deux cles distinctes mais avec le MEME hashCode
45         ClePerso k1 = new ClePerso("Awa", 42);
46         ClePerso k2 = new ClePerso("Moussa", 42);
47         map.put(k1, "contact A");
48         map.put(k2, "contact B");
49         // Les deux doivent etre retrouvables malgre la collision
50         assertEquals("contact A", map.get(k1));
51         assertEquals("contact B", map.get(k2));
52         assertEquals(2, map.size());
53     }
54
55     /** Classe auxiliaire : hashCode = second argument. Permet de
56      * forcer des collisions pour les tests. */
57     static class ClePerso {
58         final String nom;
59         final int h;
60         ClePerso(String nom, int h) { this.nom = nom; this.h = h; }

```

```

61     @Override public int hashCode() { return h; }
62     @Override public boolean equals(Object o) {
63         if (!(o instanceof ClePerso)) return false;
64         ClePerso other = (ClePerso) o;
65         return this.nom.equals(other.nom) && this.h == other.h;
66     }
67 }
68 }

```

```
gradle --no-daemon test
```

Vérification 1 — tous les tests passent

1. Les 5 tests doivent passer (BUILD SUCCESSFUL).
2. Le test `testForcerUneCollision` est le plus intéressant : il prouve que votre chaînage fonctionne. *Comment?* Les deux clés `k1` et `k2` ont le même `hashCode` (42), donc le même `h`, donc elles sont dans la même case. Votre `get` doit quand même les distinguer grâce à `equals`.

6 Étape 5 — Ajouter le redimensionnement (rehash)

Ce qu'on matérialise du CM

Facteur de charge $\alpha = n/m$ et **rehash** à $\alpha > 0,75$: quand la table devient trop pleine, on double m et on recalcule toutes les positions. Coût amorti $\mathcal{O}(1)$.

Modifiez `ChainedHashMap.java` pour ajouter la constante de seuil et la méthode `rehash` :

```

1     private static final double SEUIL_REHASH = 0.75;
2
3     @SuppressWarnings("unchecked")
4     private void rehash() {
5         LinkedList<Paire<K, V>>[] ancienneTable = table;
6         m = 2 * m; // double la taille
7         table = new LinkedList[m];
8         for (int i = 0; i < m; i++) {
9             table[i] = new LinkedList<>();
10        }
11        n = 0;
12        // Reinserer chaque paire (h(cle) sera recalculée)
13        for (LinkedList<Paire<K, V>> seau : ancienneTable) {
14            for (Paire<K, V> p : seau) {
15                put(p.cle, p.valeur);
16            }
17        }
18    }

```

Puis modifiez `put` pour déclencher le rehash *après* insertion :

```

1     public V put(K cle, V valeur) {
2         int i = hash(cle);
3         for (Paire<K, V> p : table[i]) {
4             if (p.cle.equals(cle)) {
5                 V ancienne = p.valeur;
6                 p.valeur = valeur;
7                 return ancienne;
8             }

```

```

9      }
10     table[i].add(new Paire<>(cle, valeur));
11     n++;
12
13     // Verifier le facteur de charge
14     if ((double) n / m > SEUIL_REHASH) {
15         rehash();
16     }
17     return null;
18 }

```

Piège fréquent

$n / m > 0.75$ avec deux entiers fait une division *entière* qui donne toujours 0 (ou 1 une fois la table pleine). Il faut impérativement caster : `(double) n / m > 0.75`.

6.1 Test du rehash

Ajoutez ce test dans `ChainedHashMapTest.java` :

```

1  @Test
2  void testRehashAutomatique() {
3      ChainedHashMap<Integer, String> map = new ChainedHashMap<>();
4      // Capacite initiale = 8
5      assertEquals(8, map.capacity());
6
7      // Insérer 7 paires ( $\alpha = 7/8 = 0.875 > 0.75 \rightarrow$  déclenchement)
8      for (int i = 0; i < 7; i++) {
9          map.put(i, "v" + i);
10     }
11     // Après le 7eme put, un rehash doit avoir eu lieu
12     assertEquals(16, map.capacity(), "m doit avoir double");
13     assertEquals(7, map.size(), "n doit rester a 7");
14
15     // Toutes les cles doivent encore etre accessibles apres rehash
16     for (int i = 0; i < 7; i++) {
17         assertEquals("v" + i, map.get(i));
18     }
19 }

```

```
gradle --no-daemon test
```

Vérification 2 — le rehash fonctionne

Le test `testRehashAutomatique` doit passer. Avant d'avancer, assurez-vous bien que : (a) la capacité double exactement au bon moment, (b) toutes les paires restent accessibles après le rehash.

7 Étape 6 — Observer : programme d'observation

Ce qu'on matérialise du CM

Évolution du facteur de charge et des rehashs. En CM on a annoncé qu'avec un rehash à 0,75 le facteur de charge reste borné. Ici on va visualiser cette évolution avec un programme qui affiche la capacité, la taille et α après chaque insertion.

Modifiez `src/main/java/Main.java` :

```

1 public class Main {
2     public static void main(String[] args) {
3         ChainedHashMap<Integer, String> map = new ChainedHashMap<>();
4
5         System.out.println("\n\tm\talpha\tevenement");
6         int m_precedent = map.capacity();
7
8         for (int i = 1; i <= 30; i++) {
9             map.put(i, "valeur" + i);
10            int m_actuel = map.capacity();
11            String evenement = (m_actuel != m_precedent) ? "REHASH" : "";
12            System.out.printf("%d\t%d\t%.3f\t%s\n",
13                             map.size(),
14                             m_actuel,
15                             map.loadFactor(),
16                             evenement);
17            m_precedent = m_actuel;
18        }
19    }
20 }
```

Exécution :

```
gradle --no-daemon run
```

Vous devriez obtenir un affichage ressemblant à ceci :

n	m	alpha	evenement
1	8	0.125	
2	8	0.250	
3	8	0.375	
4	8	0.500	
5	8	0.625	
6	8	0.750	
7	16	0.438	REHASH
8	16	0.500	
...			
12	16	0.750	
13	32	0.406	REHASH
...			
24	32	0.750	
25	64	0.391	REHASH
...			

Vérification 3 — observations à faire

1. Combien de rehashs se produisent pendant ces 30 insertions ? Notez les valeurs de n où ils se déclenchent : _____

2. Juste après chaque rehash, quelle est la valeur approximative de α ? _____
3. Le facteur de charge dépasse-t-il jamais 0,75 à long terme ? _____

Ce que vous venez de voir en direct

Vous venez de visualiser **empiriquement** le résultat théorique du CM : le rehash maintient α sous 0,75 *en moyenne*. Les pics dépassent brièvement 0,75 (juste avant chaque rehash) mais sont immédiatement corrigés. C'est ce qui permet de garantir $\mathcal{O}(1)$ amorti.

8 Étape 7 — Observer : distribution des seaux

Ce qu'on matérialise du CM

Uniformité du hachage : en CM on a supposé que les clés se répartissent uniformément. Ici on va vérifier que c'est bien le cas avec des chaînes de caractères typiques (via `String.hashCode()`).

Ajoutez une méthode d'inspection à `ChainedHashMap` :

```

1  /** Retourne un tableau indiquant la longueur de chaque seau.
2  *  Utile pour vérifier la distribution. */
3  public int[] bucketSizes() {
4      int[] tailles = new int[m];
5      for (int i = 0; i < m; i++) {
6          tailles[i] = table[i].size();
7      }
8      return tailles;
9  }
```

Ajoutez un second scénario dans `Main.java` (après le premier) :

```

1  System.out.println("\n--- Distribution des seaux ---");
2  ChainedHashMap<String, Integer> contacts = new ChainedHashMap<>()
3      ;
4  String[] noms = {
5      "Awa Diop", "Moussa Ndiaye", "Fatou Sall", "Cheikh Ba",
6      "Aminata Fall", "Ibrahima Kane", "Mariama Sow", "Ousmane Ba",
7      "Khadija Diallo", "Mamadou Cisse", "Aicha Gueye", "Babacar
8      Seck"
9  };
10 for (int i = 0; i < noms.length; i++) {
11     contacts.put(noms[i], i);
12 }
13
14 int[] tailles = contacts.bucketSizes();
15 System.out.println("Capacite : " + contacts.capacity()
16     + ", nb contacts : " + contacts.size()
17     + ", alpha : " + contacts.loadFactor());
18 for (int i = 0; i < tailles.length; i++) {
19     System.out.printf("Seau [%d] : %d paire(s)%n", i, tailles[i])
20     ;
21 }
```

Vérification 4 — observation de la distribution

Relancez le programme. Observez la sortie.

1. Combien de seaux ont plus d'une paire (collision visible)? _____
2. Combien de seaux sont vides? _____
3. La longueur du seau le plus rempli dépasse-t-elle 3? _____

Si la distribution vous paraît *très* inégale (par exemple un seau avec 5 paires et les autres vides), c'est que `String.hashCode()` de Java a produit des valeurs mal réparties *pour ces noms particuliers*. Cela peut arriver avec peu de clés. Ajoutez d'autres noms et relancez : la distribution s'égalise généralement.

9 Git

```
cd /workspace/lab-s03
git init
git add .
git commit -m "Lab S03 : ChainedHashMap avec rehash et observation"
git log --oneline
```

Deuxième partie

Théorie : exercices d'application

Comment remplir cette partie

Les exercices suivants sont à faire *sur papier* (ou directement dans les cases de ce document). Ils ne sont pas des *tests* : ils servent à vérifier que vous avez bien compris les mécanismes qu'on vient d'implémenter. Si vous bloquez, revenez à votre code — souvent, il contient la réponse.

10 Exercice 1 — Calcul de positions de hachage

Soit une `ChainedHashMap<String, String>` de taille $m = 7$. On vous donne les `hashCode` de 5 clés :

Clé	<code>hashCode()</code>	$h(\text{clé}) = \text{hashCode()} \bmod 7$
"Awa Diop"	-2 045 123 461	
"Moussa Ndiaye"	879 654 325	
"Fatou Sall"	1 234 567 892	
"Cheikh Ba"	-555 888 777	
"Aminata Fall"	42	

Aide : $2\,045\,123\,461 \bmod 7 = 3$; $879\,654\,325 \bmod 7 = 4$; $1\,234\,567\,892 \bmod 7 = 5$; $555\,888\,777 \bmod 7 = 3$; $42 \bmod 7 = 0$.

Question : quelles clés collisionnent ? _____

11 Exercice 2 — Trace de chaînage séparé

Reprenez les hachages calculés à l'exercice 1. Les paires sont insérées dans l'ordre :

1. `put("Awa Diop", "77 555 01 23")`
2. `put("Moussa Ndiaye", "76 123 45 67")`
3. `put("Fatou Sall", "77 888 12 34")`
4. `put("Cheikh Ba", "70 111 22 33")`
5. `put("Aminata Fall", "77 666 77 88")`

Question : dessinez l'état final. L'insertion dans une `LinkedList` se fait **en fin de liste** (l'ordre d'apparition dans la liste = ordre d'insertion).

Case	Liste chaînée (du début vers la fin)
[0]	
[1]	
[2]	
[3]	
[4]	
[5]	
[6]	

Question subsidiaire : combien de comparaisons pour `get("Aminata Fall")` ? _____

12 Exercice 3 — Trace de sondage linéaire

Mêmes clés et hachages qu'à l'exercice 2, mais cette fois avec une `LinearProbingMap` de taille $m = 7$ (un seul tableau simple, sans listes chaînées).

Règle du probing : quand la case $h(\text{clé})$ est occupée, on essaie $h + 1$, puis $h + 2$, etc. (modulo 7), jusqu'à une case libre.

(a) Remplissez le tableau après toutes les insertions :

Case	[0]	[1]	[2]	[3]	[4]	[5]	[6]
Clé (ou -)							
Indice h d'origine							
Nombre d'étapes de probing							

(b) Combien de cluster(s) (amas contigu de cases occupées) observez-vous ? _____

(c) On cherche `get("Ibrahima Kane")` avec $h = 3$. Décrire la séquence d'indices visités :

Indices visités : _____

Résultat (trouvé / absent) : _____

13 Exercice 4 — Facteur de charge et rehash

Lien avec votre implémentation

Cet exercice demande de prédire *sur le papier* exactement ce que votre programme de l'étape 6 a affiché. Vérifiez vos réponses avec la sortie console.

Une `ChainedHashMap` démarre à $m = 8$. Seuil de rehash : $\alpha > 0,75$. On insère des clés **toutes distinctes**.

(a) Combien de clés peut-on insérer *avant* que le 1^{er} rehash ne se déclenche ? (Autrement dit : à quelle insertion k le rehash se produit-il ?)

Calcul du seuil : $n > 0,75 \times 8 =$ _____

Donc le rehash se déclenche à la $k =$ _____ ème insertion.

(b) Après ce rehash, taille m' : _____. Nouveau facteur de charge $\alpha = n/m' =$ _____.

(c) Combien d'insertions supplémentaires avant le 2^e rehash ? _____

(d) Après 100 insertions successives, quelle est la taille m finale du tableau ? Donnez la suite des rehashs (à quelles valeurs de n) :

14 Exercice 5 — Contrat equals ↔ hashCode

Voici une classe `Contact` utilisée comme clé :

```

1 public class Contact {
2     private final String nom;
3     private final String numero;
4
5     public Contact(String nom, String numero) {
6         this.nom = nom;
7         this.numero = numero;
8     }
9
10    @Override
11    public boolean equals(Object o) {
12        if (!(o instanceof Contact)) return false;
13        Contact c = (Contact) o;
14        return this.nom.equals(c.nom); // egalite sur le nom seul
15    }
16
17    // hashCode() NON redefini -> herite d'Object !
18 }

```

(a) Rappeler le contrat Java entre `equals` et `hashCode` : _____

(b) On crée deux contacts avec le même nom mais différents numéros : `c1 = new Contact("Awa Diop", "77 111 22 33")` et `c2 = new Contact("Awa Diop", "77 999 88 77")`.

`c1.equals(c2)` vaut _____. `c1.hashCode() == c2.hashCode()` est _____.

Pourquoi `map.put(c1, "A"); map.get(c2);` retourne-t-il `null` ? (2-3 lignes)

(c) Écrire la méthode `hashCode()` manquante qui respecte le contrat :

```

@Override
public int hashCode() {
    _____
}

```

15 Pour aller plus loin (optionnel)

Si vous avez fini et que vous voulez creuser :

- Modifiez le seuil de rehash dans votre code (ex : 0,5 ou 0,9). Relancez `Main` et observez : les rehashs sont-ils plus fréquents ou moins fréquents ? La distribution des seaux change-t-elle ?
- Implémentez `LinearProbingMap` (sondage linéaire) en reprenant le squelette mais sans listes chaînées. Comparez les performances avec `ChainedHashMap` en ajoutant dans `Main` une boucle de 100 000 insertions chronométrée.
- Ajoutez une méthode `longestChain()` qui retourne la longueur du seau le plus rempli. Observez son évolution au fil des insertions.

16 Aide-mémoire

Aide-mémoire S03

Concept	Résumé
Fonction de hachage	$h(k) = k.hashCode() \bmod m$
Chaînage séparé	Chaque case = <code>LinkedList<Paire></code>
Sondage linéaire	Si la case est prise, essayer $(i + 1) \bmod m$
Facteur de charge	$\alpha = n/m$
Seuil de rehash	$\alpha > 0,75 \Rightarrow$ doubler m , recalculer h
Coût amorti put	$\mathcal{O}(1)$ malgré les rehashs occasionnels
Contrat Java	$a.equals(b) \Rightarrow a.hashCode() = b.hashCode()$

Commandes utiles

Commande	Usage
<code>gradle -no-daemon build</code>	Compiler le projet
<code>gradle -no-daemon test</code>	Exécuter les tests JUnit
<code>gradle -no-daemon run</code>	Exécuter la classe Main
<code>git add . && git commit -m "msg"</code>	Committer les changements

Rapport individuel

Rapport individuel — obligatoire

Chaque étudiant rédige un rapport de **1 à 2 pages** :

1. **Ce que j'ai fait** : classes implémentées, tests écrits, ce que Main affiche.
2. **Les observations clés** : qu'est-ce que le programme d'observation m'a montré que je n'avais pas bien compris du CM ?
3. **Les difficultés** : erreurs de compilation, bugs, blocages sur les exercices théoriques.
4. **Ce que j'ai appris** : le concept le plus important retenu de cette séance.

Format : PDF. **Nom** : rapport-lab-S03-PRENOM-NOM.pdf